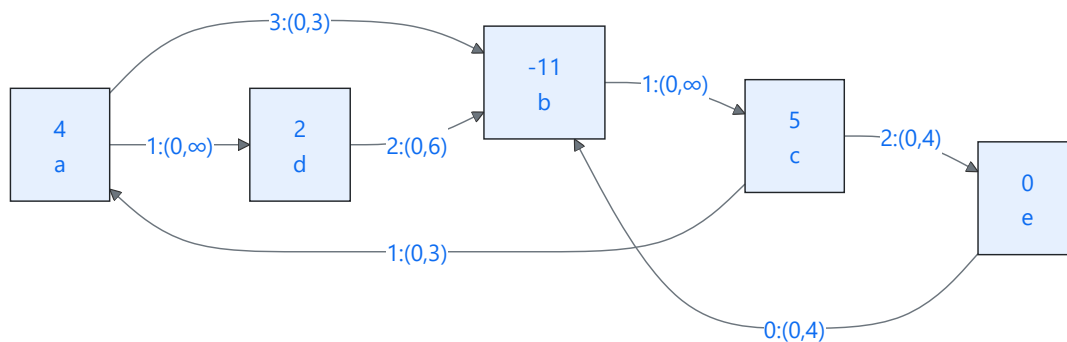
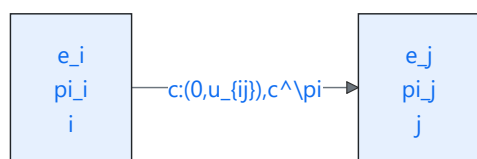


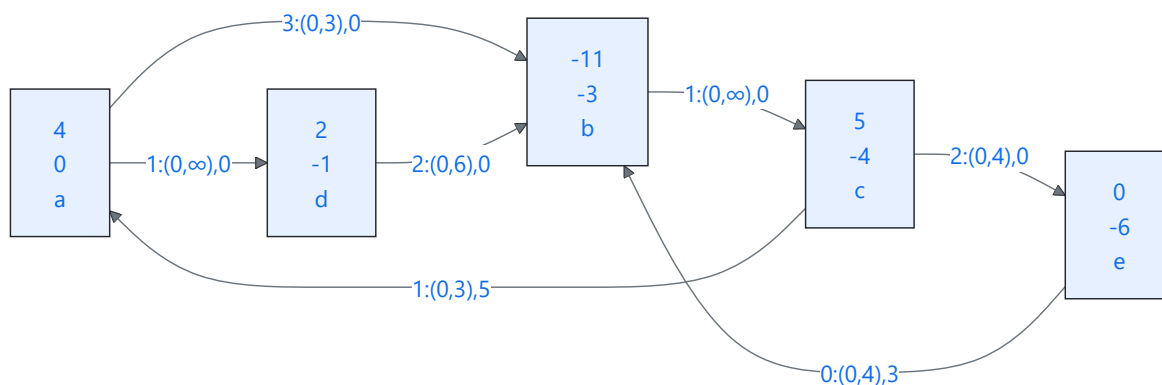
HW5 Q4



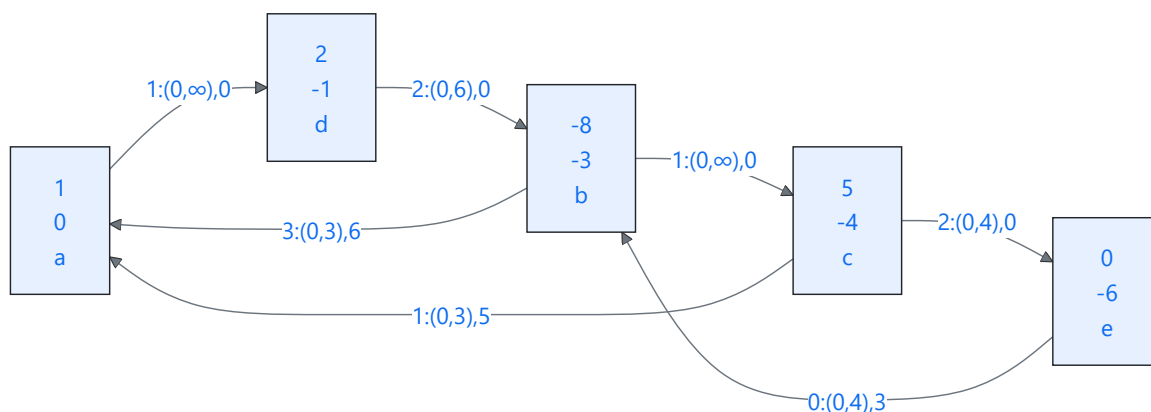
with notation as:



If we start with $s = a$, thus $\pi_a = 0$, and $t = b$ and $x = 0$ we get the below $G(x)$ and calculating $c_{ij}^{\pi} = c_{ij} - \pi_i + \pi_j = c_{ij} + d_i - d_j$:

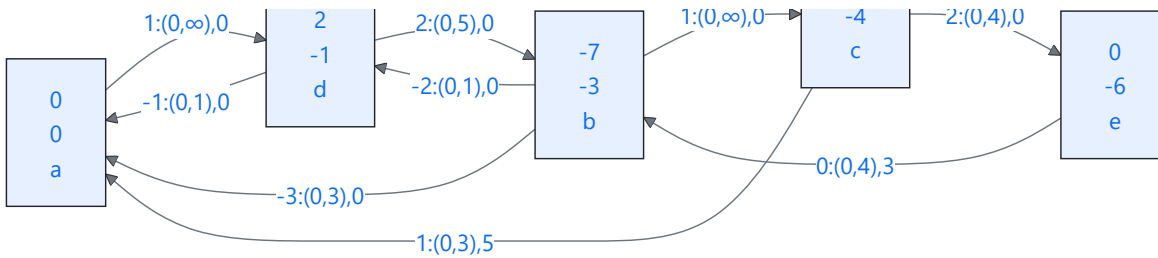


Here we see there are 2 shortest paths from a to b so we can start by just taking x_{ab} . On this path, the most amount of flow we can send is 3 because of the upper bound so let's update x, e_i and $G(x)$

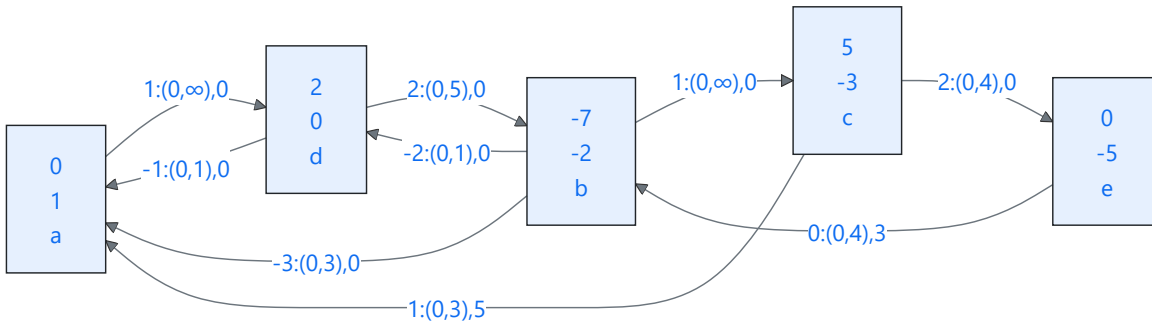


Since a still has excess and b still has deficit we repeat again this time along the path $a \rightarrow d \rightarrow b$. Here we only send 1 unit of flow because that bottoms out all of the excess at a .

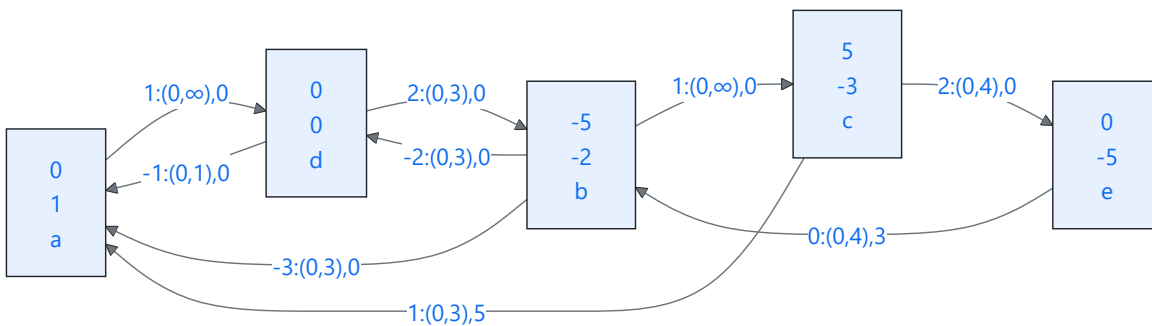




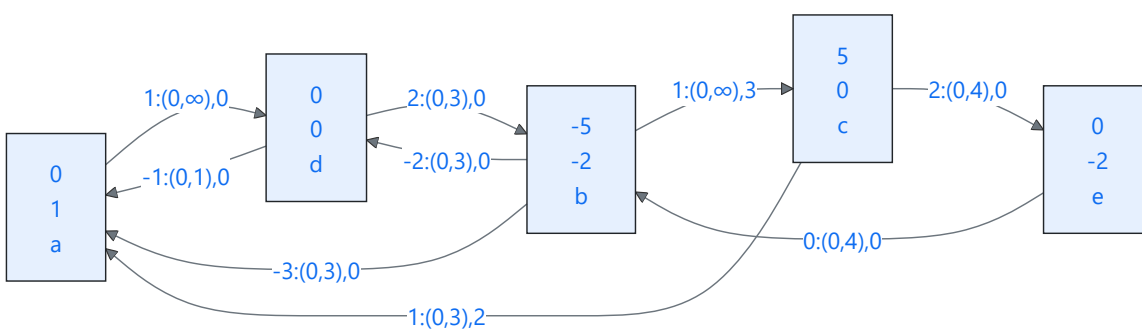
Now let's choose $s = d$ and $t = b$ and recalculate our π 's and c_{ij}^π :



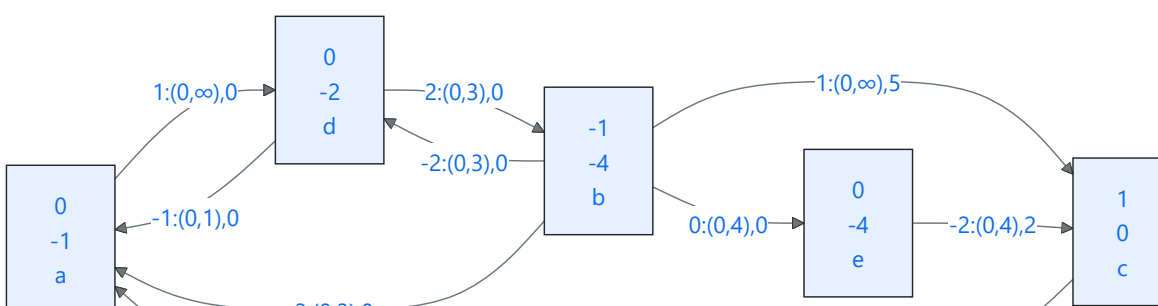
Thus we find the shortest path from d to b is $d \rightarrow b$ so we send the max amount of flow which is 2 units due to d having an excess of 2. Update:



Now let's choose $s = c$ and $t = b$ and recalculate our π 's and c_{ij}^π :



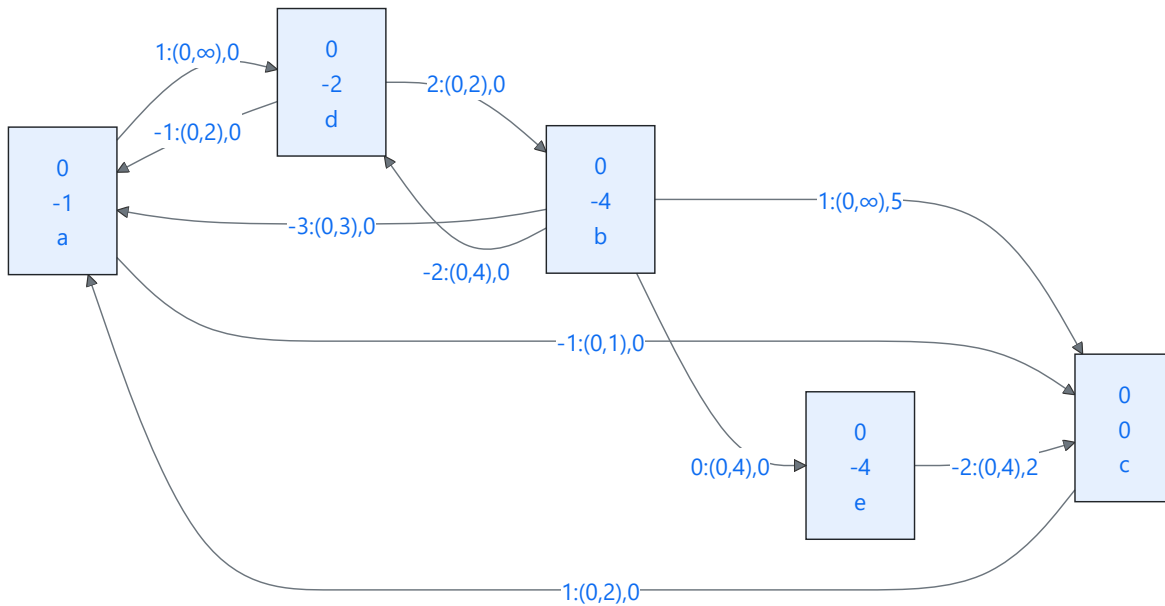
Thus we find the shortest path from c to b is $c \rightarrow e \rightarrow b$ so we send the max amount of flow which is 4 units due to arcs having an upperbound of 4. Update π_i , e^i , c_{ij}^π , $G(x)$:



$$-5:(0,3),0$$

$$1:(0,3),0$$

Thus we make our final updates with the shortest path now from c to b being $c \rightarrow a \rightarrow d \rightarrow b$ with the final remaining flow of 1. Updating everything results in:



Thus using the SSP algorithm we get the following optimal path flows solution for the network:

Path	Flow Amount
(a,b)	3
(a,d,b)	1
(d,b)	2
(c,e,b)	4
(c,a,d,b)	1

1

Let player 2 be the minimizing player who chooses strategies $S^+ = \{1, 2, 3, 4\}$, with x_1, x_2, x_3, x_4 from the LP, x_i corresponding to probability strategy i is chosen for player 2. This problem is that player 2 must reveal their mix strategy, to minimize the objective function assuming that player 1 (the maximizing player) is playing optimally. Player 1 be the maximizing player with $S^- = \{1, 2, 3, 4\}$ since there are 4 different terms in the objective function corresponding to the different choices player 1 has to make. Breaking apart the first term in the objective, it is given that $3x_1 + x_2 + x_3 + x_4$ which just means this is the expected value of $h(x, j)$ given that $j = 1$ and x is the probability distribution over S^- . Thus we can formulate our objective $h(x, y)$ as shown in the table below:

$S^- S^+$	1	2	3	4
1	3	3/2	1	1
2	1	2	4/3	4/3
3	1	1	5/3	5/3
4	1	1	1	2

Thus the LP is simply that the minimizing player is revealing their mixed strategy and then the maximizing player uses an optimal pure strategy based on the minimizing mixed strategy and the objective $h(x, y)$ shown above.

2

$$h(i, j) = \frac{b(i, j)}{a(i)} \text{ and } a(i) = \min_j b(i, j).$$

Thus $h(i, j) = \frac{b(i, j)}{a(i)} = \frac{b(i, j)}{\min_j b(i, j)}$

a

Given $v^* = \min_y \max_i h(i, y)$

By vonNeuman theorem: $v^* = \min_y \max_i h(i, y) = \max_x \min_j h(x, j)$

Since this has a $\max_x$ we know $v^* \geq \min_j h(x, j)$

b

Knowing off with $v^* = \max_x \min_j h(x, j)$, lets expand on our given expectations:

$$\frac{E_x[b(i, j)]}{E_x[a(i)]} = \frac{\sum_i x(i)b(i, j)}{\sum_k x(k)a(k)}$$

We know we don't have a divide by 0 error because $b(i, j) \geq a(i) > 0$. Thus if we are able to find a mixed strategy x' that satisfies this equality then we would have this be equal to $h(x', j)$ which is less than or equal to v^*

Let $x'(i) = \frac{x(i)a(i)}{\sum_k x(k)a(k)}$, then:

$$h(x', j) = \sum_i x'(i)h(i, j) = \sum_i \frac{x(i)a(i)}{\sum_k x(k)a(k)} \frac{b(i, j)}{a(i)} = \sum_i \frac{x(i)b(i, j)}{\sum_k x(k)a(k)} = \frac{E_x[b(i, j)]}{E_x[a(i)]}$$

We can also trivially check that $\sum_i x'(i) = 1$ by seeing that the numerator and denominator are equal after adding the summations.

Thus, since by **2.a** we know that $\forall x, v^* \geq \min_j h(x, j)$ and we found an x' as a valid x that satisfies the constraints, we know that:

$$v^* \geq \min_j h(x', j) = \min_j \frac{E_x[b(i, j)]}{E_x[a(i)]}$$

c

We can most simply recognize $\frac{1}{E_x[\frac{1}{h(i, j)}]}$ as $\frac{1}{\sum_i x(i) \frac{1}{h(i, j)}}$. This can be rewritten as a weighted harmonic mean:

$$\frac{\sum_i x(i)}{\sum_i x(i) \frac{1}{h(i, j)}} = \frac{1}{\sum_i x(i) \frac{1}{h(i, j)}}$$

This weighted harmoonic mean is less than or equal to the arithmetic mean as $\sum_i x(i)h(i, j)$

Thus putting this together:

$$\min_j \frac{1}{E_x[\frac{1}{h(i, j)}]} = \min_j \frac{1}{\sum_i x(i) \frac{1}{h(i, j)}} \leq \min_j \sum_i x(i)h(i, j) = \min_j h(x, j) \leq v^*$$

With the last inequality from **2.a**. Thus all **2.a, 2.b, 2.c** are all true inequalities

3

a

Start with node assignments where x_7 be the middle node and x_1 be the left node in the top section and we can count up going clockwise with the node down and to the left of x_1 being x_6 and the node to the right of x_1 being x_2 .

We create our linear program as follows:

$$\min \sum x$$

$$\text{s.t. } x_1 + x_2 \geq 1$$

$$x_3 + x_4 \geq 1$$

$$x_5 + x_6 \geq 1$$

$$x_1 + x_7 \geq 1$$

$$x_2 + x_7 \geq 1$$

$$x_3 + x_7 \geq 1$$

$$x_4 + x_7 \geq 1$$

$$x_5 + x_7 \geq 1$$

$$x_6 + x_7 \geq 1$$

$$x \geq 0$$

And dual as:

$$\max \sum y$$

$$\text{s.t. } y_{12} + y_{17} \leq 1$$

$$y_{12} + y_{27} \leq 1$$

$$y_{34} + y_{37} \leq 1$$

$$y_{34} + y_{47} \leq 1$$

$$y_{56} + y_{57} \leq 1$$

$$y_{56} + y_{67} \leq 1$$

$$y_{17} + y_{27} + y_{37} + y_{47} + y_{57} + y_{67} \leq 1$$

$$y \geq 0$$

So we initialize our PD algorithm with $H = \phi$ and $y = 0$

Say we start with the edge (1, 2), so we increase y_{12} to 1 and append nodes 1, 2 to H .

In our next iteration we can pick edge (3, 4) and increase y_{34} and append nodes 3, 4 to H .

In our final iteration we can pick edge (5, 6) (or any remaining edge as long as they don't include a node within our set) and increase y_{56} and append nodes 5, 6 to H .

Thus we get our final vertex cover as vertexes 1, 2, 3, 4, 5, 6

b

We create our linear program as done in **3.a**

It is clear that all $x^* = \frac{1}{2}$ as the optimal solution since it results in an objective of 3.5 which is less than any optimal integer solution.

Now by calculating $(1 - x_i^*)^3 = \frac{1}{2}^3 = 0.125$ and $\frac{1}{e} \approx 0.3678794412$. Thus since $\frac{1}{e}$ is always greater than or equal to $(1 - x_i^*)^3$ we never omit any edge.

Thus we end up with a feasible vertex cover solution of including every node, since every edge is covered because if an edge is not covered it implies having a node that is not part of the graph which makes no sense, but this is likely not an optimal solution.

c

Vertex cover as BLP is:

$$\min \sum x$$

$$\text{s.t. } x_i + x_j \geq 1, ((i, j) \in E)$$

$$\forall x, x \in \{0, 1\}$$

When formulating this as a QUBO we only need to relax the constraints $x_i + x_j \geq 1, ((i, j) \in E)$. This is a simple or constraint which can be written as $1 - x_i - x_j + x_i x_j$.

Thus formulating this as QUBO is the lagrangian relaxation as follows:

$$\min \sum_{i=1}^n x_i + \lambda \sum_{(i,j) \in E} (1 - x_i - x_j + x_i x_j)$$

If we want the QUBO specific to this problem then we get the following:

$$\min \sum_{i=1}^n x_i + \lambda(9 - 2x_1 - 2x_2 - 2x_3 - 2x_4 - 2x_5 - 2x_6 - 6x_7 + x_1x_2 + x_1x_7 + x_2x_7 + x_3x_4 + x_3x_7 + x_4x_7 + x_5x_6 + x_5x_7 + x_6x_7)$$

4

a

As given in **4.b** that a feasible α is negative, which is problematic for SDP since we cannot have negative variables. We can solve this by having α be denoted as the difference between two variables α^+ and α^- . Additionally, SDP in standard form cannot have inequalities but only equalities so we need to add slack variables s .

Essentially, since we are given that our current X is positive semi definite, we can leave X as is but add a few more rows and columns to our extended X which I'll denote as X' .

Thus X' is as follows:

$$X' = \begin{pmatrix} X & 0 & 0 & 0 \\ 0 & \alpha^+ & 0 & 0 \\ 0 & 0 & \alpha^- & 0 \\ 0 & 0 & 0 & I_s \end{pmatrix}$$

We leave all constraints the same as before except for replacing α with $\alpha^+ - \alpha^-$ and adding the slack variables s_i to the i th constraint.

Additionally we need to add constraints to ensure the non diagonal entries of X' are 0 (with the exception of entries in X), which are easily done by adding constraints $X'_{ij} = 0, (i \neq j \wedge (i > |A| \vee j > |A|))$

Thus our final program now looks like:

$$\min X'_{|A|+1,|A|+1} - X'_{|A|+2,|A|+2} = \alpha^+ - \alpha^-$$

$$\text{s.t. } X'_{IJ} - X'_{|A|+1,|A|+1} + X'_{|A|+2,|A|+2} + s_k = 0, (\text{if } \{I, J\} \in c(A))$$

$$X'_{II} = 1, (I \in A)$$

$$X'_{ij} = 0, (i \neq j \wedge (i > |A| \vee j > |A|))$$

$$X \succeq 0$$

note that s_k is the slack variable for the k th constraint and is only used once.

b

Essentially we must construct a feasible solution X, α with $\alpha = -\frac{1}{k-1}$ given that k is the minimum number of districts.

Since X is positive semi definite, $\exists B, B^T B = X$. Let B be $k \times |A|$ matrix corresponding to the assignments of our districts. That is a single column of B represents an arc of A and this column will have a single non zero entry corresponding to which district it is in.

This gets us a preliminary solution of $\alpha = 0$ since the dot product in the B matrix of any conflicting arcs would be 0 and $X_{II} = 1$ since $b_i^T b_i = 1$.

Consider the case of $k = 2$, instead of having our two district vectors as $[1, 0]$ and $[0, 1]$ instead we can do $[1, 0]$ for arcs in our first district and $[-1, 0]$ for arcs in the second district. This way our vectors are still unit vectors and $b_i^T b_i = 1$ and conflicting vectors (or any arcs not in the same district) i, j now results in $b_i^T b_j = -1$ which satisfies our $\alpha = -1$ for $k = 2$

For $k = 3$ we can perform the same logic but instead on an equilateral triangle vector space. This results in $b_i^T b_i$ always being 1 (because unit vectors) while we are essentially equally maximizing the distance from all other conflicting arcs.

We can do a proof by induction to ensure that the dot products of conflicting arcs follows the $-\frac{1}{k-1}$ rule. We did the base case now for the inductive step. Assuming the minimum dot product for the k vectors is $\frac{1}{k-1}$ we need to show that the minimum dot product for $k + 1$ vectors is $-\frac{1}{k}$.

If we think about this on a unit hypersphere, when going from k to $k + 1$ vectors we should make the $k + 1$ vector along a new axis and shift the rest of the vectors "down" the hypersphere by $\frac{1}{k}$. Thus its clear that the dot product of our new product with the other vectors is $-\frac{1}{k}$ but what about the other vectors. Suppose we model our original vectors as v and now after shifting them down they are now $(sv, -\frac{1}{k})$ with the s to denote a scalar multiplier to ensure that the vector remains a unit vector. Solving for s :

$$1 = \sqrt{\frac{1}{k^2} + s^2 \sum v^2} \rightarrow \sqrt{\frac{1 - \frac{1}{k^2}}{|v|^2}} = s^2 \text{ with the denominator going to 1 because } v \text{ is a unit vector already}$$

Thus we have that our new vectors are $(\sqrt{1 - \frac{1}{k^2}}v, -\frac{1}{k})$

Now calculating the dot product between any of the two updated vectors u and u' :

$$u \cdot u' = (1 - \frac{1}{k^2})v \cdot v' + \frac{1}{k^2}$$

$$\rightarrow (1 - \frac{1}{k^2})(\frac{-1}{k-1}) + \frac{1}{k^2}$$

$$\rightarrow (\frac{k^2-1}{k^2})(\frac{-1}{k-1}) + \frac{1}{k^2}$$

$$\rightarrow (\frac{(k+1)(k-1)}{k^2})(\frac{-1}{k-1}) + \frac{1}{k^2}$$

$$\rightarrow (\frac{k+1}{k^2})(-1) + \frac{1}{k^2}$$

$$\rightarrow \frac{1-k}{k^2} + \frac{1}{k^2}$$

$$\rightarrow -\frac{k}{k^2}$$

$$\rightarrow -\frac{1}{k}$$

Therefore proof by induction we are able to find $k + 1$ vectors that have dot products of $-\frac{1}{k}$.

Going back to the districts, given k minimum districts, we have shown that we could hypothetically find k vectors along this unit hypersphere which allows us to satisfy $b_i^T b_i = 1$ while any conflicting arcs would simply fall under the category of appearing in different districts resulting in $b_i^T b_j = -\frac{1}{k-1}$ as shown above. It is worth mentioning just for clarity that essentially each district is assigned a vector on this sparse unit hypersphere, and our matrix B is comprised of n columns with each column being a vector from our set of k vectors that satisfy that dot product above. Now we can number each of these vectors as $v_1, v_2, \dots, v_k$ and the column b_i has vector v_j iff the i th arc is a member of district j .